

Python 爬蟲課程

第一篇 · 數據力基礎與環境建置



邵子卿老師編製

CH1 Python 核心語法複習 + 虛擬環境

1-1 為什麼爬蟲需要虛擬環境？認識 venv

一. 觀念介紹：為什麼要用虛擬環境？

多個專案若共用同一套 Python 環境，容易發生「套件版本衝突」：

- 專案 A 使用 1.0 版套件
- 專案 B 需要 2.0 版

若直接安裝在系統中，可能導致專案壞掉。

虛擬環境 (venv) 的作用：為每個專案建立獨立環境，彼此互不影響。

二. 指令範例：建立與啟動 venv

Windows 系統

```
python -m venv venv
.\venv\Scripts\activate    # 確認是否出現 (venv)
pip install requests
```

macOS / Linux 系統

```
python3 -m venv venv
source venv/bin/activate    # 確認是否出現 (venv)
pip install requests
```

三. 指令說明

- **python -m venv venv**
呼叫 venv 模組建立虛擬環境，venv 為資料夾名稱（可自訂）。
- **activate**
啟動環境後，套件會安裝在此環境內，而非系統。
- **(venv)**
出現在終端機前方代表已進入虛擬環境。

四. 常見錯誤

2. Windows 權限錯誤
→ 使用系統管理員執行 PowerShell，設定 Set-ExecutionPolicy RemoteSigned
 3. 找不到 python
→ 安裝時需勾選「Add Python to PATH」
 4. 未啟動環境就安裝
→ 沒看到 (venv) 時安裝會污染全域環境
-

1-1 隨堂練習

一、選擇題 (5 題)

1. () 為什麼要使用虛擬環境？ (A) 讓電腦跑更快 (B) 節省硬碟空間 (C) 避免不同專案的套件版本衝突 (D) 讓 Python 介面變漂亮。
2. () 在 Windows 啟動虛擬環境的正確指令路徑是？ (A) `.\venv\activate` (B) `.\venv\Scripts\activate` (C) `source venv/bin/activate` (D) `run venv`。
3. () 建立虛擬環境後，資料夾內通常會出現什麼？ (A) 一個名為 `venv` 的資料夾 (B) 一個名為 `python.exe` 的圖示 (C) 專案的所有程式碼 (D) 自動生成的爬蟲腳本。
4. () 如何判斷目前是否已成功進入虛擬環境？ (A) 電腦會發出聲音 (B) 終端機路徑前方出現 `(venv)` (C) 資料夾會變色 (D) Python 會自動開啟。
5. () 如果想結束虛擬環境回到正常狀態，應輸入什麼指令？ (A) `stop` (B) `exit` (C) `deactivate` (D) `close`。

二、填充題 (5 題)

1. 建立虛擬環境的指令是 `python -m ____ venv`。
2. 在 macOS 或 Linux 系統，啟動環境需要使用 ____ 指令。
3. 我們通常使用 ____ 來安裝 Python 的第三方套件。
4. 若要查看目前虛擬環境安裝了哪些套件，可以輸入 `pip ____`。
5. 虛擬環境資料夾的名稱雖然可以自訂，但社群最通用的慣例名稱是 ____。

三、實作題 (2 題)

1. 任務一：請在桌面建立一個資料夾 `crawler_test`，並在該資料夾內建立一個虛擬環境，啟動它後截圖終端機畫面（需看到 `(venv)` 字樣）。
2. 任務二：啟動虛擬環境後，請安裝 `requests` 套件與 `pandas` 套件，並使用 `pip list` 指令列出安裝清單。

1-2 套件管理：pip 安裝、requirements.txt 紀錄環境

一. 觀念介紹：開發者的「備忘錄」

虛擬環境建立後，需要安裝套件。

requirements.txt 的用途：

記錄專案所需套件與版本，例如：

- requests==2.31.0
- pandas==2.1.1

讓他人能快速還原相同環境。

二. 指令範例：安裝、紀錄與還原

確保已經「啟動」了 1-1 建立的虛擬環境 (venv)：

1. 安裝套件

`pip install requests` # 也可以同時安裝多個套件

`pip install pandas beautifulsoup4`

2. 產生清單 (匯出環境)

`pip freeze > requirements.txt`

3. 一鍵還原 (根據清單安裝)

`pip install -r requirements.txt`

三. 指令說明

- `pip`：Python 套件管理工具 (從 PyPI 下載套件)
- `pip freeze`：列出目前所有套件與版本
- `>`：將輸出結果寫入檔案
- `requirements.txt`：純文字套件清單
- `pip install -r`：依清單逐行安裝

四. 常見錯誤

1. 找不到 `pip` → 未啟動 `venv` 或 `PATH` 未設定
 2. 檔名錯誤 → 必須是 `requirements.txt`
 3. 網路逾時 → 可改用鏡像源 (通常台灣可直接使用)
 4. 忘記加「`>`」 → 只會顯示，不會存檔
-

1-2 隨堂練習

一、選擇題 (5 題)

1. () pip install 的主要用途是什麼？ (A) 編寫 Python 程式碼 (B) 刪除電腦上的檔案 (C) 下載並安裝第三方套件 (D) 重新啟動電腦。
2. () 如果要把目前的套件清單存成檔案，應該使用哪個指令？ (A) pip list (B) pip freeze > requirements.txt (C) pip install requirements (D) pip save。
3. () 在 pip install -r requirements.txt 中，參數 -r 代表什麼意思？ (A) remove (移除) (B) run (執行) (C) read (讀取檔案) (D) repair (修復)。
4. () pip freeze > requirements.txt 指令執行後，檔案會出現在哪裡？ (A) 雲端硬碟 (B) C 槽根目錄 (C) 目前操作的資料夾下 (D) Python 安裝路徑。
5. () 為什麼 requirements.txt 裡面通常會包含版本號（例如 requests==2.31.0）？ (A) 為了讓檔案看起來比較專業 (B) 避免未來套件更新導致舊程式碼失效 (C) 這是作業系統規定的 (D) 讓檔案變大。

二、填充題 (5 題)

1. Python 官方提供的套件管理工具簡稱叫作 ____。
2. 指令中使用的 ____ 符號可以將輸出的內容寫入到一個文字檔中。
3. 若要查看目前環境安裝了哪些套件（不一定要存檔），可以使用 pip ____。
4. 慣例上，記錄專案套件需求的清單檔案名稱為 _____.txt。
5. 若要移除一個已安裝的套件，指令是 pip _____ <套件名稱>。

三、實作題 (2 題)

1. **任務一：**請在虛擬環境中安裝 requests 和 lxml 兩個套件，然後使用指令產出 requirements.txt。請打開該檔案，確認裡面是否有這兩個套件及其版本號。
2. **任務二：**請手動建立一個名為 test.txt 的檔案，內容寫入 pyquery（這也是一種爬蟲套件），然後嘗試使用 pip install -r test.txt 來安裝它。最後用 pip list 確認是否安裝成功。

1-3 複習：List 與 Dict 的巢狀結構處理

重要觀念：爬蟲的數據流水線 (Data Pipeline)

爬蟲資料處理流程：

1. 抓取 (Crawl)
2. 清洗 (Clean)
3. 儲存 (Store)
4. 分析 (Analyze)

本節重點：清洗 → 儲存前的資料結構整理

1-3-1：List (列表) — 數據的排隊序列

一. 觀念介紹

List 是「有順序的集合」，常用來存放同類資料：

- 例如：新聞標題、價格清單
- 編號從 0 開始
- 可巢狀 (List 裡面放 List)

二. 程式範例：處理多筆新聞資料

```
news_titles = ["台積電股價新高", "AI 概念股轉強", "美股四大指數收紅"]
news_data = [ # [標題, 點擊數]
    ["台積電股價新高", 1500],
    ["AI 概念股轉強", 800],
    ["美股四大指數收紅", 1200]
]
# 第一則新聞的點擊數
click_count = news_data[0][1]
print(f"第一則新聞：{news_data[0][0]}")
print(f"點擊數為：{click_count}")
```

三. 程式說明

- news_data[0]：取得第一筆資料 (子 List)
- news_data[0][1]：取得點擊數
- 常用於簡單表格資料

四. 常見錯誤

- IndexError：索引超出範圍 (注意從 0 開始)
-

1-3-2 : Dictionary (字典) — 具備標籤的資料

一. 觀念介紹

Dict 是「Key-Value 結構」：

優點：語意清楚（例如 price: 100）

二. 程式範例：描述一則新聞

```
single_news = {  
    "title": "台積電股價新高",  
    "clicks": 1500,  
    "source": "財經日報"  
}
```

```
print(f"新聞標題：{single_news['title']}")  
print(f"來源是：{single_news.get('source')}")
```

三. 程式說明

- dict['key']：直接取值
- .get()：安全取值（不存在時回傳 None）

四. 常見錯誤

- KeyError：存取不存在的 key
→ 建議使用 .get()

1-3-3 : 巢狀結構 (List of Dicts) — 爬蟲數據的標準格式

一. 觀念介紹

最常見結構：List + Dict

- List：多筆資料
- Dict：單筆資料的欄位
- 常見於 API / JSON

二. 程式範例：模擬 API 抓回來的多筆資料

這就是最典型的爬蟲數據結構

```
all_news = [  
    {"title": "台積電", "price": 800, "rank": 1},  
    {"title": "聯發科", "price": 1000, "rank": 2},  
    {"title": "鴻海", "price": 150, "rank": 3}  
]
```

練習：迴圈搭配巢狀存取

```
print("--- 今日股市熱搜 ---")
for stock in all_news:
    # stock 在每一輪會代表一個字典
    name = ???
    price = ???
    print(f"名稱: {name} | 價格: {price}")
```

三. 程式說明

- all_news 是 List → 用 for 迴圈遍歷
- stock 是 Dict → 用 key 存取
- 可直接轉為 Pandas 表格

四. 常見錯誤

- 型態混淆，記住：
外層 List (用索引) + 內層 Dict (用 key)

1-3 補充範例：List 與 Dict 的多樣化應用

核心：API 資料解析基礎

範例一：購物車清單 (List 內含 Dict + 數值計算)

常見於電商資料 (如蝦皮、Momo) 總額計算。

```
cart = [
    {"name": "Python 書籍", "price": 450, "count": 1},
    {"name": "無線滑鼠", "price": 890, "count": 2},
    {"name": "螢幕支架", "price": 1200, "count": 1}
]
```

```
total_cost = 0
for item in cart:
    subtotal = item["price"] * item["count"]
    total_cost += subtotal
    print(f'商品: {item["name"]}, 小計: {subtotal}')

print(f'總結帳金額: {total_cost}')
```

範例二：班級成績單 (Dict 內含 List)

以主鍵 (學生) 對應多筆資料 (成績)。

```
grades = {  
    "Alice": [85, 90, 78],  
    "Bob": [70, 65, 80],  
    "Charlie": [92, 88, 95]  
}
```

```
# 計算 Alice 的平均分  
alice_avg = sum(grades["Alice"]) / len(grades["Alice"])  
print(f"Alice 的平均分是: {alice_avg:.2f}")
```

1-3 隨堂練習

一、選擇題 (5 題)

1. () 關於 List 的編號，下列何者正確？ (A) 從 1 開始 (B) 從 0 開始 (C) 隨機編號 (D) 由字母 A 開始。
2. () 如果想在字典中取得資料且「避免標籤不存在時報錯」，建議使用哪個方法？ (A) list() (B) print() (C) .get() (D) .find()。
3. () data = [{"name": "Apple"}, {"name": "Banana"}]，請問 data[1]["name"] 會得到什麼？ (A) Apple (B) Banana (C) name (D) 報錯。
4. () 在爬蟲流程設計中，「清洗」的主要目的是？ (A) 讓網頁載入變快 (B) 刪除電腦病毒 (C) 從雜亂字串提取出結構化資料 (D) 讓程式碼變短。
5. () 字典 (Dict) 的組成結構是？ (A) Index 與 Value (B) Key 與 Value (C) List 與 Tuple (D) Row 與 Column。

二、填充題 (5 題)

1. 在 Python 中，列表使用 [] 括號，而字典使用 ____ 括號。
2. 巢狀結構 stocks = [{"TSMC", 800}, {"Foxconn", 150}] 中，要取得 150 這個數字的寫法是 stocks[1][____]。
3. 若要遍歷一個裝有字典的列表，通常會配合使用 ____ 迴圈。
4. 字典中，左邊的標籤稱為 ____，右邊的資料稱為 Value。
5. 爬蟲標準流程是：抓取 → 清洗 → ____ → 分析。

三、實作題 (2 題)

1. **任務一：資料提取** 現有一個抓回來的資料如下： site_data = {"city": "Taipei", "weather": [{"time": "morning", "temp": 25}, {"time": "night", "temp": 18}]} 請寫出程式碼，印出 "Taipei 晚上的氣溫是 18 度"。(提示：層層拆解，先取字典，再取列表中的字典)。
2. **任務二：結構建立** 請手動建立一個變數 my_favorites，格式為 List 內裝 3 個 Dict。每個 Dict 須包含 title (書名) 與 price (價格) 兩個標籤。最後用 for 迴圈印出所有書名。

1-4 實戰：撰寫第一個自動化檔案備份腳本

一. 觀念介紹：為什麼爬蟲需要學備份？

常見問題：

- 資料被覆蓋（重跑程式）
- 無法保留不同時間版本（HTML 比對）

備份流程：

1. 檢查檔案是否存在
2. 加上時間戳記避免重名
3. 複製到目標位置

使用模組：

- os：路徑與檔案檢查
- shutil：檔案操作

二. 程式範例：自動時間戳記備份

```
import os
import shutil
from datetime import datetime

def backup_file(source_file, target_folder):

    if not os.path.exists(source_file):
        print(f'錯誤：找不到來源檔案 {source_file}')
        return

    if not os.path.exists(target_folder):
        os.makedirs(target_folder)
        print(f'建立新資料夾：{target_folder}')

    try:
        # 產生時間戳記檔名 (例如: 20260413_1530_data.csv)
        timestamp = datetime.now().strftime("%Y%m%d_%H%M")
        file_name = os.path.basename(source_file)
        new_file_name = f'{timestamp}_{file_name}'
        # 組合完整的目標路徑
        destination = os.path.join(target_folder, new_file_name)
        shutil.copy2(source_file, destination) # 執行複製
        print(f'備份成功！檔案已存至：{destination}')
```

except Exception as e:

```
print(f"備份過程中發生未知錯誤：{e}")
```

```
backup_file("data.csv", "my_backups")
```

三. 程式說明

- `os.path.exists()`：檢查檔案存在，避免錯誤（提升穩定性）
- `os.makedirs()`：建立多層資料夾
- `datetime.now().strftime()`：產生時間字串（年/月/日/時/分）
- `os.path.basename()`：取得檔名
- `shutil.copy2()`：複製檔案並保留中繼資料
- `os.path.join()`：跨平台安全組合路徑（避免手動拼接）

四. 常見錯誤

1. `PermissionError`
 - 檔案被其他程式（如 Excel）佔用
 2. 檔名非法字元
 - 避免使用 `:`、`\`、`/`，改用 `_` 或 `-`
 3. 路徑斜線問題（Windows）
 - 使用 `r"路徑"` 或 `/`
-

1-4 補充範例：自動化檔案處理與備份

範例一：日誌輪替 (Log Rotation)

將舊 log 移動到 archive，避免主目錄堆積。

```
import os
import shutil

source = "today_spider.log"
target_dir = "history_logs" # 模擬將今天的爬蟲記錄移到 archive 資料夾
if not os.path.exists(target_dir):
    os.makedirs(target_dir)
if os.path.exists(source):
    shutil.move(source, os.path.join(target_dir, "old_20260413.log")) # 移動並改名
    print("舊日誌已歸檔。")
```

範例二：批次重新命名照片 (整理素材)

抓取圖片後，統一命名方便管理。

```
import os

folder = "scraped_images" # 假設資料夾裡有 img1.jpg, img2.jpg...
files = os.listdir(folder) # 列出資料夾內所有檔案
for index, filename in enumerate(files):
    # 建立新名字：pet_0.jpg, pet_1.jpg...
    new_name = f"pet_{index}.jpg"
    old_path = os.path.join(folder, filename)
    new_path = os.path.join(folder, new_name)
    os.rename(old_path, new_path)
    print(f"更名成功: {filename} -> {new_name}")
```

Sect 1-4 實戰：撰寫第一個自動化檔案備份腳本

一、選擇題 (5 題)

1. () 下列哪一個模組主要用於高階檔案操作，例如：複製整個檔案或保留檔案中繼資料？
(A) math (B) shutil (C) random (D) json。
2. () 在 Windows 系統中，路徑通常使用反斜線 \，為了避免轉義字元報錯，我們可以在字串前加上哪個字母？ (A) s (B) f (C) r (D) b。
3. () 使用 `datetime.now().strftime("%Y%m%d")`，若今天是 2026 年 4 月 13 日，輸出的結果會是什麼？ (A) 2026/04/13 (B) 20260413 (C) 13-04-2026 (D) 2026 April 13。
4. () 哪一個指令可以用來合併資料夾路徑與檔案名稱，且能自動根據作業系統調整斜線方向？ (A) `os.path.add()` (B) `os.path.join()` (C) `os.path.merge()` (D) `shutil.combine()`。
5. () 為什麼在執行複製前，建議先使用 `os.path.exists()`？ (A) 讓程式跑得更快 (B) 節省硬碟空間 (C) 確保來源檔案存在，避免程式因錯誤而崩潰 (D) 這是 Python 的法律規定。

二、填充題 (5 題)

1. 建立多層資料夾 (如果不存在就建立) 的指令是 `os._____`。
2. 若只想從完整路徑 `C:/data/test.csv` 中取出檔名 `test.csv`，可以使用 `os.path._____`。
3. 在 `strftime` 格式化中，代表「小時 (24 小時制)」的代號是 `%____`。
4. 當備份過程中發生檔案被其他程式開啟而無法存取時，會觸發 `_____Error`。
5. 寫程式時，利用 `try..._____` 結構可以捕捉異常，防止程式直接中斷。

三、實作題 (2 題)

1. **任務一：簡單備份腳本** 請撰寫一段程式碼，將目前目錄下的 `notes.txt` 複製到一個名為 `old_notes` 的資料夾內，並將備份後的檔名改為 `backup_notes.txt`。(不需加時間戳記，只需簡單複製並改名)。
2. **任務二：雙重安全檢查備份** 請修改課堂上的範例，增加一個檢查：「如果目標資料夾內已經存在同名的備份檔，則印出『檔案已存在，跳過備份』；否則才執行複製動作。」

1-5 開發工具實務：VS Code 與 Git 入門

一. 觀念介紹：為什麼需要 Git？

Git 可視為程式碼的「時光機」與「保險箱」：

- 時光機：可回到過去版本
- 保險箱：GitHub 提供雲端備份，避免資料遺失

二. VS Code 強力外掛推薦

提升開發效率的三個工具：

- Python (Microsoft)：語法提示、補完、除錯
- GitLens：顯示每行程式的修改紀錄（適合協作）
- Thunder Client：模擬 API 請求，無需寫程式即可測試

三. Git 核心指令「四部曲」

標準流程：

1. `git init`：初始化版本控制
2. `git add .`：加入暫存區
3. `git commit -m "訊息"`：建立版本快照
4. `git push`：同步到 GitHub

提醒：每完成一小段就 `commit`，是良好習慣。

四. 安全第一：.gitignore 與隱私保護

用途：避免上傳敏感資料或大型檔案

實作：

1. 建立 `.gitignore`
2. 設定排除項目：
排除虛擬環境
`venv/`
`.env`
排除機密金鑰檔案
`config.json`
`api_key.txt`
排除暫存檔
`__pycache__/`
`.DS_Store`

五. GitHub 建立遠端倉庫

1. 建立 repository (不勾 README)
2. 綁定本地專案：
 - `git remote add origin [網址]`
 - `git branch -M main`
 - `git push -u origin main`

1-5 補充：從 GitHub 取回與還原資料

一. 第一次取回專案：git clone

- 指令：
`git clone [GitHub 倉庫網址]`
- 效果：下載完整專案與歷史紀錄

二. 同步最新進度：git pull

- 指令：
`git pull origin main`
- 效果：更新並合併最新版本

三. 救命恩人：發生災難時的「還原」

情境 A：還原單一檔案

`git checkout -- [檔名]`

或

`git restore [檔名]`

情境 B：全部還原 (不可逆)

`git reset --hard HEAD`

四. Git 指令完整流程圖 (進階版)

方向	動作	指令	記憶法
Local → Remote	上傳	<code>add → commit → push</code>	存錢
Remote → Local	下載	<code>pull / clone</code>	領錢
Undo	還原	<code>checkout / reset</code>	時光機

1-5 隨堂練習

一、實作

任務 1：建立你的第一個版控專案

1. 在電腦建立資料夾 `my_python_course`。
2. 使用 VS Code 開啟，並建立一個 `hello.py` 寫入 `print("Hello Git")`。
3. 開啟終端機，執行 `git init`。
4. 建立 `.gitignore` 檔案，輸入 `secret.txt`。
5. 執行 `git add`，與 `git commit -m "第一次提交"`。
6. (加分題) 在 GitHub 建立同名倉庫，並成功 `push` 上去。

任務 2：體會「時光機」的神奇

1. 開啟上次建立的 `hello.py`，故意亂寫一些會報錯的亂碼，並存檔。
2. 執行 `git status`，你會看到紅色的訊息告訴你檔案被修改了。
3. 執行 `git checkout -- hello.py` (或 `git restore hello.py`)。
4. 回到 VS Code 查看：剛才的亂碼消失了，程式碼回到了上次存檔的狀態！

二、觀念問答

1. () 如果不小心把資料夾刪除了，在哪個前提下我們可以救回程式碼？ (A) 有裝 Python (B) 有執行過 `commit` (C) 有裝 VS Code。
2. () 若要排除特定檔案不被 Git 追蹤，應該在哪個檔案裡設定？ (A) `hello.py` (B) `.gitignore` (C) `settings.json`。
3. () 如果我想把 GitHub 上「別人的專案」整個下載下來，應該用哪個指令？ (A) `git pull` (B) `git push` (C) `git clone`。
4. () 發現程式碼大改爛，想「強行回溯」到最後一次成功 `commit` 的狀態，應使用？ (A) `git reset --hard HEAD` (B) `git undo` (C) `git del`。

CH1 Python 核心語法與環境建置：課後綜合練習

一、選擇題 (15 題)

1. () 下列哪一項是使用虛擬環境 (venv) 的主要優點？ (A) 增加 CPU 運算速度 (B) 隔離不同專案的套件版本 (C) 自動備份所有程式碼 (D) 讓 VS Code 執行得更流暢。
2. () 在 Windows 終端機執行 `python -m venv myenv` 後，會發生什麼事？ (A) 安裝最新版 Python (B) 執行一個名為 `myenv` 的程式 (C) 在當前目錄建立一個名為 `myenv` 的環境資料夾 (D) 刪除舊有的環境。
3. () 啟動 Windows 虛擬環境的正確指令為： (A) `source venv/bin/activate` (B) `venv/activate` (C) `.\venv\Scripts\activate` (D) `python start venv`。
4. () 想要快速安裝 `requirements.txt` 中所列的所有套件，應該輸入： (A) `pip install requirements.txt` (B) `pip freeze > requirements.txt` (C) `pip install -r requirements.txt` (D) `python install -all`。
5. () `pip freeze > requirements.txt` 中的 `>` 符號作用是？ (A) 比較大小 (B) 啟動程式 (C) 指向路徑 (D) 將結果重導向輸出至檔案。
6. () 列表 `a = [10, 20, 30]`，請問 `a[1]` 是多少？ (A) 10 (B) 20 (C) 30 (D) 報錯。
7. () 字典 `d = {"price": 100}`，哪種方式取值最安全（標籤不存在時不會當掉）？ (A) `d["price"]` (B) `d.get("price")` (C) `d.find("price")` (D) `d.show("price")`。
8. () 爬蟲流程「抓取 → 清洗 → 儲存 → 分析」，本章 CH1 練習的自動備份腳本主要屬於哪一環？ (A) 抓取 (B) 清洗 (C) 儲存 (D) 分析。
9. () 下列哪種資料結構最適合描述「一本書的書名、作者、價格」？ (A) List (B) Dictionary (C) String (D) Integer。
10. () `shutil.copy2()` 比 `shutil.copy()` 強大的地方在於？ (A) 速度快十倍 (B) 會保留檔案的最後修改時間等中繼資料 (C) 可以複製網頁網址 (D) 不需要匯入模組。
11. () `os.path.join("data", "test.csv")` 在 Windows 會產生什麼結果？ (A) `data/test.csv` (B) `datatest.csv` (C) `data\test.csv` (D) `data+test.csv`。
12. () 若要檢查資料夾 `backup` 是否存在，應使用？ (A) `os.exists("backup")` (B) `os.path.exists("backup")` (C) `shutil.exists("backup")` (D) `os.check("backup")`。
13. () `datetime.now().strftime("%Y_%m")` 會輸出什麼格式？ (A) `2026/04` (B) `2026_04` (C) `Monday_April` (D) `04_2026`。
14. () 在 Python 中，若想捕捉所有可能導致程式中斷的錯誤，應使用： (A) `if...else` (B) `for...in` (C) `try...except` (D) `while...True`。
15. () 執行 `pip install requests` 後，該套件會被存放在？ (A) 虛擬環境中的 `site-packages` 資料夾 (B) 桌面 (C) C 槽根目錄 (D) 我的文件。

二、填充題 (10 題)

1. 當我們在終端機提示字元前看到 (_____) 字樣時，代表已進入虛擬環境。
2. 卸載 (結束) 虛擬環境的指令是 _____。
3. 在字典中，{"name": "Gemini"} 裡的 "name" 稱為 _____ (鍵)。
4. os.path._____ 指令可以只取路徑最後的檔案名稱，捨棄資料夾路徑。
5. 若要建立一個多層的資料夾路徑，建議使用 os._____。
6. pip _____ 指令可以列出目前環境中安裝的所有套件名稱與版本。
7. List 結構中，若有 5 個元素，最後一個元素的索引編號 (正數) 是 _____。
8. 在路徑字串前加上字母 _____ 可防止 \ 被視為轉義字元 (如：r"C:\temp")。
9. 在 strftime 中，代表「分鐘」的格式符號是 %_____。
10. shutil 模組全名與其功能相關，其含義是 Shell _____。

三、實作題 (2 題)

任務一：學生成績管理與搜尋系統 (難度：★★)

情境：你有一份學生資料，格式如下：

Python

```
students = [  
    {"id": "A01", "name": "Alice", "score": 85},  
    {"id": "A02", "name": "Bob", "score": 92},  
    {"id": "A03", "name": "Charlie", "score": 78}  
]
```

要求：

1. 撰寫一個迴圈，計算所有學生的 **平均分數** 並印出。
2. 撰寫一個搜尋功能：讓使用者輸入學生姓名。
 - 若姓名存在，印出該學生的 ID 與分數。
 - 若姓名不存在，印出「查無此人」。

任務二：進階日誌備份機器人 (難度：★★★)

情境：模擬爬蟲執行後的 Log 檔案管理。 要求：

1. 在專案中建立一個名為 logs 的資料夾，並在裡面手動建立一個 spider.log。
 2. 撰寫一個 Python 程式，執行時會執行以下動作：
 - 讀取 spider.log 的內容並印在螢幕上。
 - 在 backups 資料夾內建立一個以「今天日期」命名的子資料夾 (例如 backups/20260413/)。
 - 將 spider.log 複製到該子資料夾中，並在檔名後方加上小時與分鐘 (例如 spider_2130.log)。
 - 備份完成後，將原始的 logs/spider.log 刪除 (提示：搜尋 os.remove)，模擬檔案移轉過程。
 3. 加入 try...except 結構，確保即使 logs 資料夾裡沒有檔案，程式也不會崩潰，而是印出「目前沒有日誌需要處理」。
-